

Anvil RTL Bug Study

Root-Cause Analysis, Reproduction, and Detection of Real SystemVerilog Bugs

Author: Intikhab Khursheed | **Date:** September 2026

Repository: github.com/IntikhabKhursheed/anvil-rtl-bug-study

1. Introduction

1.1 The Problem

Hardware designs written in SystemVerilog can contain bugs that survive code review, linting, and simulation. Many of these bugs arise from recurring failure patterns such as handshake violations, parameter mismatches, and incomplete case handling. They can remain hidden because the conditions required to trigger them are often not covered by ordinary testing.

This study examines five real, documented RTL bugs and asks which of their failure classes can be prevented by Anvil's type system and which remain outside its guarantees.

1.2 What I Did

I searched public issue trackers, pull requests, commits, and release notes to identify five real hardware bugs that satisfy the study requirements. For each bug, I examined its RTL-level mechanism, the reason it could survive existing verification, its broader bug class, and appropriate detection methods such as SVA, linting, formal verification, and simulation.

Three bugs were reproduced in standalone SystemVerilog simulation. The reproductions compare buggy and corrected implementations and use assertions to detect the violated behavior. All three reproducers also include an Anvil encoding, compiled with Anvil compiler revision d138cabedbf3b65c08249ce6a55cb90dad959da, with testbenches that check the generated designs under back-pressure.

1.3 The Five Bugs

Bug	Design	Class	Anvil
1	OpenTitan keymgr_dpe	Width / parameterization	No
2	iDMA	Handshake-gated state advancement	Yes
3	common_cells FIFO	Handshake-gated state advancement	Partial
4	CVFPU	Incomplete case / functional omission	No
5	OpenTitan EDN	Handshake-gated state advancement	Yes

1.4 Key Finding

Bugs 2, 3 and 5 belong to the handshake-gated state advancement class. The iDMA and OpenTitan EDN cases both involve valid/ready communication under back-pressure but arise through different RTL mechanisms. For communication expressed through Anvil channels, Anvil's blocking send/rcv semantics prevent the specific failure mechanisms observed in both cases.

Bug 3 sits on the boundary. A FIFO write pointer advanced on valid_i alone without a completed handshake. Anvil's channel sequencing makes the correct ordering natural but does not prove all FIFO bookkeeping correct, so the verdict is partial.

Bugs 1 and 4 fall outside this boundary entirely. Bug 1 involves a width mismatch established at elaboration time. Bug 4 involves a functional case omission that Anvil's type system does not track. These failures require linting, assertions, formal verification, or functional testing. Anvil complements these techniques rather than replacing them.

1.5 Report Structure

Sections 2 through 6 examine the five bugs individually. Each section covers the original instance, RTL mechanism, why the bug survived, its general class, detection methods, and Anvil's applicability. Section 7 presents the reproducers. Section 8 compares results across all five bugs and discusses the boundary between what Anvil enforces and what other techniques must cover.

2. Bug 1 — OpenTitan keymgr_dpe Width Truncation

Source: Issue #25994, PR #26055 | Class: Width/Parameterization | Anvil: No

The keymgr_dpe block derives cryptographic key material across DPE contexts and sends advance data to KMAC for processing. The keymgr and keymgr_dpe modules shared a single AdvDataWidth parameter set to 1152 bits. The DPE path required 1664 bits to accommodate additional slot data, so 512 bits were silently lost before reaching KMAC. Issue #25994 identifies the dropped fields as sw_binding and revision_seed. PR #26055 lets keymgr_dpe override the shared AdvDataWidth parameter with its own value, while the regular keymgr keeps its default.

Simplified extract of the real code, which is a size-cast concatenation (AdvDataWidth'({...})) in the adv_matrix assignment:

```
// Before: shared parameter is 1152 bits, concatenation is 1664 bits
parameter int AdvDataWidth = 1152;
adv_matrix[Creator] = AdvDataWidth'({sw_binding, revision_seed, ..., creator_seed}); // 512 bits lost

// After: keymgr_dpe overrides AdvDataWidth
parameter int AdvDataWidth = 1664; // keymgr_dpe only
```

The size cast silently truncates the concatenation without a runtime error. The issue reports that the omission was found by reading the RTL, not through a failing test.

This bug belongs to the width/parameterization class. The violated invariant is that every consumer of a shared parameter must be independently verified to have compatible width requirements. A related example from the exclusion list is verilog-axi #44, where address-decode logic assumed a minimum address width without enforcing it.

Width-mismatch lint checks and elaboration-time checks are the appropriate detection tools. Runtime SVA is not the primary mechanism here because the failure is established structurally at elaboration, not at runtime.

Anvil does not prevent this bug. The failure is a parameterization problem outside Anvil's timing-safety model.

3. Bug 2 — iDMA Error Handler Valid/Ready FSM Violation

Source: PR #93, merged 2026-05-21 | Class: Handshake-gated state advancement | Anvil: Yes

The iDMA engine handles direct memory transfers and includes an error handler that emits a response when a transfer fails. The response path has a valid/ready interface: the handler asserts rsp_valid_o to offer a response, and the receiver asserts rsp_ready_i to accept it.

The FSM unconditionally left its IDLE state, in which the response was offered, without checking whether the receiver was ready. On the following cycle, rsp_valid_o fell to zero because the FSM had already advanced. If the receiver had not yet asserted rsp_ready_i, the response was lost. The following is a simplified extract isolating the mechanism:

```
// Simplified extract – before fix
rsp_valid_o = 1;
```

```
state_d = WAIT; // transitions without checking ready

// Simplified extract – after fix
rsp_valid_o = 1;
if (rsp_ready_i) state_d = WAIT; // held until handshake completes
```

The bug appears only when the receiver applies back-pressure (`rsp_ready_i` low) in the cycle the response is offered. The same pull request also fixed a second, separate omission: a missing `eh_valid_i` check in `WAIT_LAST_W`.

This bug belongs to the handshake-gated state advancement class. The violated invariant is that protocol-visible state may advance only after its corresponding valid/ready transfer completes. Independent instances appear in Bug 3 and Bug 5, where different state types advance without a completed transfer, and in `pulp-platform/axi`, where B and R payloads changed between valid and the handshake (exclusion list, cited as a supporting instance).

The following SVA property targets the specific observed violation:

```
assert property (@(posedge clk) disable iff (!rst_n)
  (rsp_valid_o && !rsp_ready_i) | => rsp_valid_o)
else $error("rsp_valid_o retracted before handshake");
```

The property checks valid stability only. A fuller property would additionally verify that the response payload remains stable while valid is asserted and ready is low.

For communication expressed through Anvil channels, the `send >>` set state sequencing prevents the state transition from occurring before the receiver accepts the response. The Anvil compiler generated synchronization state that holds `rsp_valid` asserted during back-pressure, which was confirmed by running the generated design under three cycles of forced back-pressure during which the SVA property never fired.

4. Bug 3 — `common_cells_passthrough_stream_fifo` Pointer Advancement

Source: PR #322, merged 2026-06-30, Issues #264 and #313 | Class: Handshake-gated state advancement | Anvil: Partial

The `common_cells_passthrough_stream_fifo` is a low-latency FIFO that avoids cutting the timing path and supports simultaneous push and pop. PR #322 fixed a bug where the write pointer advanced on `valid_i` alone and the read pointer advanced on `ready_i` alone. A push while the FIFO was full could therefore alter `wr_ptr_q` even though no transfer completed. The PR states that with assertions disabled, including in synthesized netlists, this corrupted the pointers whenever a push happened while full or a pop happened while empty. The only guards were two simulation-only `ASSERT_NEVER` checks (`CheckFullPush` and `CheckEmptyPop`); the fix removed them because the corrected logic makes those states unreachable. The fix was merged as commit `a49dc11`.

```
// Before – pointer advances on valid_i alone
if (in_valid_i) wr_ptr_q <= wr_ptr_q + 1;

// After – pointer advances only on completed handshake
if (in_valid_i && in_ready_o) wr_ptr_q <= wr_ptr_q + 1;
```

The read pointer fix applies the same principle: `rd_ptr_q` advances only on `ready_i && valid_o`.

The bug required a push while the FIFO was full, or a pop while it was empty, and was visible only in assertion-enabled simulation.

This bug belongs to the handshake-gated state advancement class. The violated invariant is that protocol-visible state may advance only after its corresponding valid/ready transfer completes. In `iDMA` an FSM state advanced early; in `EDN` a payload advanced early; here a FIFO pointer advanced early. All three let state progress without a completed transfer, making `iDMA` PR #93 and `OpenTitan` EDN #15469 independent instances of the same broad class.

The SVA property detects the write-side violation:

```
assert property (@(posedge clk) disable iff (!rst_n)
  (in_valid_i && !in_ready_o) | => $stable(wr_ptr_q))
else $error("FIFO write pointer advanced without handshake");
```

Anvil partially reduces this risk. If the FIFO is written as `recv >> advance_write_pointer`, the pointer cannot advance before the input transfer completes. However Anvil does not know that an arbitrary register is a FIFO pointer and does not prohibit a designer from updating that register before the channel operation. To test this I wrote two small Anvil sources. `fifo_push.anvil` sequences the pointer update after the send; in the generated `FifoWriter` module the update sits inside the branch gated by the acknowledge signal, so the ordering is enforced structurally, and the generated design held `wr_ptr` stable under three cycles of forced back-pressure. `fifo_push_gap.anvil` updates the state with no channel operation at all, and it compiles with no error or warning, because there is nothing for Anvil to check. The verdict is therefore PARTIAL: channel sequencing makes the correct implementation natural but does not make every incorrect FIFO bookkeeping implementation impossible.

5. Bug 4 — CVFPU FMA ADDS Incomplete Case Handling

Source: PR #122, merged 2024-05-23 | Class: Incomplete Case/Functional Omission | Anvil: No

CVFPU is a parametric floating-point unit used in PULP-platform processors. It supports standard RISC-V floating-point operations plus transprecision extensions. PR #114 added a new symmetric add operation ADDS to the operation enum. PR #122 is the subsequent bug fix.

When ADDS was introduced, three case statements in the FMA datapath were not updated to handle it. The affected locations were the operand default assignment in `fpnew_fma.sv`, the same case in `fpnew_fma_multi.sv`, and the addend exponent rebias logic which used `dst_fmt_i` instead of `src_fmt_i` for the ADDS path. When ADDS was requested, all three fell through to the default branch, producing zero operands or incorrect format extraction. The developer confirmed the omission in the PR description.

```
// Before – ADDS falls through to default
case (op_i)
  fpnew_pkg::ADD: begin ... end
  fpnew_pkg::MUL: begin ... end
  // ADDS not present
  default: begin op_a = '0; op_b = '0; op_c = '0; end
endcase

// After – ADDS handled explicitly
case (op_i)
  fpnew_pkg::ADD: begin ... end
  fpnew_pkg::MUL: begin ... end
  fpnew_pkg::ADDS: begin ... end
  default: begin op_a = '0; op_b = '0; op_c = '0; end
endcase
```

The bug survived because existing regression tests covered only the operations present before PR #114. The ADDS path was never exercised, and the default branch produced no compile-time error.

This bug belongs to the incomplete case/functional omission class. The violated invariant is that every valid value of an operation enum must be handled explicitly in every case statement that dispatches on it. A related example from the exclusion list is Ibex #1018, where reserved instruction encodings were incorrectly accepted rather than trapped. It is cited as a supporting example only and is not claimed to have the identical mechanism as this bug.

Detection requires functional coverage rather than structural checking alone. The unique case construct causes a simulation warning if any value reaches no branch, and formal case-coverage analysis can prove all enum values are handled. Operation-specific regression tests remain important because a missing branch may go unnoticed until the corresponding operation is exercised.

Anvil does not prevent this bug. The failure is a functional logic omission rather than a timing hazard. A case statement with a missing branch compiles and runs in Anvil provided the timing is safe.

6. Bug 5 — OpenTitan EDN Valid/Ready Back-pressure Violation

Source: Issue #15469, PR #15478 (merge commit 8d42791), PR #15402 (unmerged historical hot-fix) | Class: Handshake-gated state advancement | Anvil: Yes

The OpenTitan Entropy Distribution Network sends multiword RES/GEN commands to the CSRNG block. The interface uses a valid/ready handshake: EDN asserts `csrng_req_valid` and places command data on `csrng_req_bus`, and CSRNG asserts `csrng_req_ready` when it accepts the word.

Before the fix, `edn_core.sv` drove the CSRNG interface directly from its advancing internal command registers:

```
// Before – bus driven directly from advancing registers
assign csrng_cmd_o.csrng_req_valid = cs_cmd_req_vld_out_q;
assign csrng_cmd_o.csrng_req_bus = cs_cmd_req_out_q;
```

When `csrng_req_ready` was low, EDN continued advancing its internal state and updating `cs_cmd_req_out_q`. The offered word on `csrng_req_bus` therefore changed before CSRNG accepted it. CSRNG waited for the missing words and EDN waited for the final acknowledgement, leaving both blocks stalled. Issue #15469 reports that specific seed words were absent from the received sequence as a result.

PR #15478 introduced a synchronous output FIFO between EDN and CSRNG. The FIFO pops only when both valid and ready are asserted:

```
// After – output held in FIFO until handshake completes
assign csrng_cmd_o.csrng_req_valid = sfifo_output_not_empty;
assign csrng_cmd_o.csrng_req_bus = sfifo_output_rdata;
assign sfifo_output_pop =
    sfifo_output_not_empty && csrng_cmd_i.csrng_req_ready;
```

The bug required `csrng_req_ready` to be low during a multiword command. The CSRNG test agent did not randomize ready during such sequences, and PR #15402 explicitly noted that no test existed to verify this behavior.

This bug belongs to the same handshake-gated state advancement class as Bugs 2 and 3. The failure mechanism differs: iDMA retracted valid entirely, the common_cells FIFO advanced a pointer, while EDN kept valid asserted but mutated the payload. All three violated the same invariant: state must not advance before the corresponding transfer completes. The pulp-platform/axi B and R payload instability (exclusion list) is a further supporting instance of the payload-mutation form.

The SVA property targets the observed violation:

```
assert property (@(posedge clk) disable iff (!rst_n)
    (csrng_req_valid && !csrng_req_ready) | =>
    (csrng_req_valid && $stable(csrng_req_bus)))
else $error("EDN changed or withdrew request under back-pressure");
```

For communication expressed through Anvil channels, the `send >> advance` sequencing prevents EDN from updating its command source before CSRNG accepts the current word. This was verified by compiling the Anvil encoding and running the generated design under forced back-pressure, during which the SVA property never fired.

7. Reproducers

A reproducer is a standalone compilable SystemVerilog design that isolates the failure mechanism of a specific bug and makes it observable through simulation. Each reproducer contains a buggy implementation, a corrected

implementation, and an SVA property that fails on the buggy version and passes on the corrected one. The two implementations are controlled by a single BUGGY parameter.

Three reproducers were built. The assignment requires at least two; all three are included because they span both positive and partial Anvil boundary cases.

All reproducers, plus the Anvil verification run, can be run with one command (Verilator 5.x required):

```
bash run_reproducers.sh
```

A non-zero exit on a buggy design is expected: it means the assertion fired (134 under Verilator on Linux, 1 on EDA Playground). The script was run end to end in a clean Ubuntu environment (Verilator 5.020), and each reproducer was also validated on EDA Playground (Verilator 5.044).

7.1 Reproducer 1 — iDMA PR #93

The reproducer models the response-emission FSM. The BUGGY=1 version advances FSM state unconditionally regardless of `rsp_ready_i`. The BUGGY=0 version guards the transition.

The testbench holds `rsp_ready_i` low for three cycles after the response is offered. On the buggy version `rsp_valid_o` drops before the handshake completes and the SVA property fires. On the fixed version `rsp_valid_o` remains asserted and the property passes.

```
verilator --binary --assert --timing -Wno-fatal idma93_reproducer.sv tb_idma93.sv --top-module  
tb_idma93  
./obj_dir/Vtb_idma93
```

The Anvil encoding was compiled with the Anvil compiler (revision d138cabedbf3b65c08249ce6a55cb90dad959da). The generated design held `rsp_valid` asserted under the same back-pressure stimulus and the SVA property never fired.

7.2 Reproducer 2 — common_cells PR #322

The reproducer fills a two-entry FIFO then presents a third word while the FIFO is full, at which point `in_ready_o` is low. The BUGGY=1 version advances `wr_ptr_q` on `valid_i` alone. The BUGGY=0 version gates the update on `valid_i` && `in_ready_o`.

The reproducer exercises only the write-side failure. The read-pointer fix gates `rd_ptr_q` on `ready_i` && `valid_o`, but the testbench does not include a pop-while-empty stimulus.

```
verilator --binary --assert --timing -Wno-fatal cc_passthrough_fifo_handshake.sv  
tb_cc_passthrough_fifo.sv --top-module tb_cc_passthrough_fifo  
./obj_dir/Vtb_cc_passthrough_fifo
```

Expected: Cycle 3 shows `buggy_wr_ptr=1`, `fixed_wr_ptr=0`, then the assertion fires on the buggy instance. Two Anvil encodings were compiled for this bug: a channel-gated version (`fifo_push.anvil`) and a version with an ungated pointer update (`fifo_push_gap.anvil`). On EDA Playground the buggy instance fails its assertion at 65000 ns, and the channel-gated Anvil design prints PASS. The gap version compiling without complaint is the evidence for the PARTIAL verdict.

7.3 Reproducer 3 — OpenTitan EDN #15469

The reproducer models the EDN to CSRNG command interface. The BUGGY=1 version drives the bus from an advancing internal register. The BUGGY=0 version uses a small output FIFO that holds the current word until `csrng_req_ready` is asserted.

The testbench sends three command words and holds `csrng_req_ready` low during the second and third cycles. On the buggy version the bus changes during back-pressure and the SVA property fires. On the fixed version the same word remains stable until `ready` is asserted.

```
verilator --binary --assert --timing -Wno-fatal edn_csrng_if.sv tb_edn_handshake.sv --top-module
tb_edn_handshake
./obj_dir/Vtb_edn_handshake
```

The Anvil encoding was compiled and the generated design passed the same back-pressure test. The SVA property never fired.

8. Closing Synthesis

8.1 Classes Found

The five bugs span three distinct classes. Bugs 2, 3, and 5 all belong to the handshake-gated state advancement class but with different Anvil verdicts depending on whether the state is expressed through an Anvil channel.

Class	Bugs	Detection	Anvil
Width/Parameterization	Bug 1	Lint, elaboration checks	No
Handshake-gated state advancement	Bugs 2, 3, 5	SVA, formal	Yes (2, 5) / Partial (3)
Incomplete case	Bug 4	unique case, coverage, regression	No

8.2 Detection Techniques

Width and parameterization errors are structural and are best caught at elaboration time through lint checks and compile-time width verification. Runtime assertions cannot substitute for structural checking because the incorrect width is established before simulation begins.

Handshake-gated state advancement violations are well served by SVA properties that check state stability while a transfer is stalled. The cost is that each property must be written for a specific state element and handshake pair. Formal verification can prove the property exhaustively but requires modeling assumptions about the environment.

This same property can be reused for any valid/ready interface, not just this one bug — you just plug in the right signal names. But it only checks that the data stays the same while waiting. It doesn't check if the data was correct in the first place.

Incomplete case handling is detected through the unique case construct, formal case coverage analysis, and operation-specific regression tests. The unique case construct causes a simulation warning when a value reaches no branch. Regression tests remain necessary because a missing branch may go unnoticed until the corresponding operation is exercised. The problem with unique case is that it only warns if a test actually triggers that value. If no test ever uses it, there's no warning — which is exactly why Bug 4 wasn't caught.

8.3 Anvil's Boundary

Bugs 2 and 5 demonstrate the full positive boundary. The iDMA error handler retracted valid before the handshake completed. The OpenTitan EDN mutated the offered payload while the receiver was applying back-pressure. For communication expressed through Anvil channels, blocking send/recv semantics and sequential composition prevent both mechanisms. This was verified experimentally by compiling Anvil encodings and running the generated designs under forced back-pressure. The SVA property never fired on either generated design.

Bug 3 demonstrates the partial boundary. Expressing the FIFO pointer update as `recv >> advance_write_pointer` would prevent this specific error. However Anvil does not know that an arbitrary register is a FIFO pointer and does not prohibit a designer from updating that register before the channel operation. Channel sequencing makes the correct implementation natural but does not make every incorrect FIFO bookkeeping implementation impossible. SVA or formal FIFO data-conservation checks remain necessary.

Bugs 1 and 4 fall outside the boundary entirely. Bug 1 is a parameterization error established at elaboration time. Bug 4 is a functional completeness problem. Neither involves a timing hazard that Anvil's type system can observe.

8.4 What Resists All Techniques

A branch that runs fine but just computes the wrong answer would pass every check here — lint, unique case, and Anvil all check that something ran or was timed safely, but none of them check if the actual result is correct.

The partial boundary shown by Bug 3 is the most instructive result. A FIFO pointer is mutable state that must be gated on a handshake, but Anvil has no way to identify it as such without the designer expressing the dependency through a channel. This shows that Anvil's protection requires the designer to use its channel model for the relevant interface, not merely to compile existing RTL with the Anvil toolchain.

8.5 Conclusion

The five bugs show that Anvil's guarantee is narrow but deep. It provides a strong static guarantee for timing safety through its type system and communication semantics, but it does not replace verification of functional correctness or parameter compatibility.

Bugs 2 and 5 demonstrate the positive boundary: both violations are prevented when the interface is expressed through Anvil channels. Bug 3 demonstrates the partial boundary: channel sequencing helps but does not prove arbitrary internal bookkeeping correct. Bugs 1 and 4 demonstrate the negative boundary: their failures concern parameterization and functional logic rather than timing safety.

Anvil is best understood as a complement to existing RTL verification techniques. It enforces valid/ready stability as a language-level property for communication expressed through its channels. For everything outside that model, SVA, formal verification, lint, and functional testing remain necessary.

9. References

Primary Bug Sources

- Bug 1 — OpenTitan `keymgr_dpe` width truncation. Issue #25994: github.com/lowRISC/opentitan/issues/25994 (accessed 2026-09-05). PR #26055: github.com/lowRISC/opentitan/pull/26055 (accessed 2026-09-05).
- Bug 2 — iDMA error handler valid/ready FSM violation. PR #93, merged 2026-05-21: github.com/pulp-platform/iDMA/pull/93 (accessed 2026-09-04).
- Bug 3 — `common_cells` `passthrough_stream_fifo` pointer advancement. PR #322, merged 2026-06-30 (commit a49dc11): github.com/pulp-platform/common_cells/pull/322 (accessed 2026-09-13). Issue #313 (also addressed by PR #322). Issue #264: github.com/pulp-platform/common_cells/issues/264 (accessed 2026-09-13).
- Bug 4 — CVFPU FMA ADDS incomplete case handling. PR #122, merged 2024-05-23: github.com/openhwgroup/cvfpupull/122 (accessed 2026-09-05).
- Bug 5 — OpenTitan EDN valid/ready back-pressure violation. Issue #15469: github.com/lowRISC/opentitan/issues/15469 (accessed 2026-09-07). PR #15478, merge commit 8d42791: github.com/lowRISC/opentitan/pull/15478 (accessed 2026-09-07). PR #15402 (unmerged historical hot-fix): github.com/lowRISC/opentitan/pull/15402 (accessed 2026-09-07).

Anvil Sources

- Yu, J. Z., Jha, A. R., Mathur, U., Carlson, T. E., Saxena, P. Anvil: A General-Purpose Timing-Safe Hardware Description Language. ASPLOS '26. [arXiv:2503.19447](https://arxiv.org/abs/2503.19447) (accessed 2026-09-08).

- AnvilHDL language reference: docs.anvil.kisp-lab.org/languageReference.html (accessed 2026-09-04).
- AnvilHDL communication guide: docs.anvil.kisp-lab.org/communication.html (accessed 2026-09-04).
- Anvil compiler revision used: d138cabedbf3b65c08249ce6a55cb90dad959da.

Supporting Instances (Exclusion List)

- verilog-axi #44. Address-decode width assumption. Supporting instance for the Bug 1 class.
- Ibex #1018. Reserved encoding accepted without trap. Supporting instance for the Bug 4 class.
- pulp-platform/axi B/R payload stability. Supporting instance for the Bug 2 and Bug 5 class.

Search and Selection

- Full search log including accepted and rejected candidates: search_log.md.
- Full source list with roles and access dates: sources.md.